

Question Bank — Week 4

Dynamic arrays and amortised analysis (Lecture 04) — Questions

DSA27 · Data Structures and Algorithms

Fall 2026

Answers are in a separate file: [week04-answers.md](#). Levels: **[what]** recall · **[how]** apply · **[why]** explain and justify. Unless a question says otherwise, a dynamic array starts **empty with capacity 1**, and “doubling” means that an append which finds it full first doubles the capacity.

Part	Type	Questions
A	Multiple choice (one correct answer of four)	W4-M01 – W4-M22
B	Short answer and essay	W4-E1 – W4-E5
C	Trace the growth — show every step	W4-T1 – W4-T5
D	Proofs	W4-P1 – W4-P2
E	Find and fix the bug	W4-B1 – W4-B4
F	Write the code — checked by pytest	W4-C1 – W4-C3

Part A — Multiple choice

W4-M01 [what] For a `DynamicArray`, `len(a)` returns:

- **a)** the size — the number of elements stored
- **b)** the capacity of the Array underneath
- **c)** capacity minus size
- **d)** the number of resizes so far

W4-M02 [how] With doubling, what is the capacity after 5 appends?

- **a)** 5
- **b)** 6
- **c)** 8
- **d)** 16

W4-M03 [how] With doubling, how many resizes do 1,000 appends cause?

- **a)** 9
- **b)** 10
- **c)** 500

- **d)** 999

W4-M04 [how] If the capacity grows by **one** slot at every resize, the total number of copies for n appends is:

- **a)** $\Theta(n)$
- **b)** $\Theta(\log n)$
- **c)** $\Theta(n \log n)$
- **d)** $\Theta(n^2)$

W4-M05 [why] If the capacity grows by a constant **100** slots at every resize, the amortised cost of append is:

- **a)** $O(n)$ — the constant only divides the copying by 100
- **b)** $O(1)$ — 100 is large enough
- **c)** $O(\log n)$
- **d)** $O(100)$, which is $O(1)$

W4-M06 [what] With doubling, the amortised cost of append is:

- **a)** $O(\log n)$
- **b)** $O(1)$
- **c)** $O(n)$
- **d)** $O(n \log n)$

W4-M07 [why] With doubling, the **worst-case** cost of a single append is:

- **a)** $O(1)$
- **b)** $O(\log n)$
- **c)** $O(n)$
- **d)** $O(n^2)$

W4-M08 [why] How does amortised cost differ from average-case cost?

- **a)** They are the same thing
- **b)** Amortised cost is always larger
- **c)** Amortised cost averages over random inputs
- **d)** Amortised cost averages over a sequence of operations and needs no assumption about the inputs

W4-M09 [how] With doubling, the total number of element **copies** for n appends is:

- **a)** less than $2n$
- **b)** exactly $n \log_2 n$
- **c)** exactly $n^2/2$
- **d)** exactly n

W4-M10 [how] With doubling, which of these appends triggers a resize?

- **a)** the 16th
- **b)** the 17th
- **c)** the 18th
- **d)** the 32nd

W4-M11 [why] To shrink without thrashing, a dynamic array should halve its capacity when the size falls to:

- **a)** half the capacity
- **b)** capacity – 1
- **c)** one quarter of the capacity
- **d)** zero

W4-M12 [what] “Thrashing” in a dynamic array means:

- **a)** the array is full
- **b)** an append causes a resize
- **c)** the garbage collector runs
- **d)** alternating operations near one boundary make the array resize on almost every operation

W4-M13 [what] CPython’s list grows its capacity by about:

- **a)** one eighth, plus a little
- **b)** one slot
- **c)** a factor of 10
- **d)** exactly double

W4-M14 [how] How can you see a Python list’s capacity from Python?

- **a)** `len(values)`
- **b)** `from sys.getsizeof(values)`
- **c)** `values.capacity`
- **d)** it cannot be observed

W4-M15 [how] In a DynamicArray of size 5 and capacity 8, `a[-1]` reads `self._block` at index:

- **a)** 7
- **b)** –1
- **c)** 4
- **d)** it raises `IndexError`

W4-M16 [what] `insert_at(0, x)` on a dynamic array of size n costs:

- **a)** $O(1)$
- **b)** amortised $O(1)$
- **c)** $O(\log n)$
- **d)** $O(n)$

W4-M17 [what] `pop()` — removing the **last** element — costs:

- **a)** $O(1)$
- **b)** $O(\log n)$
- **c)** $O(n)$
- **d)** $O(n^2)$

W4-M18 [why] Why does growing by a factor give amortised $O(1)$ append?

- **a)** Because the array never needs to copy

- **b)** Because the copies form a geometric series, dominated by its last term, which is less than n
- **c)** Because Python optimises it
- **d)** Because resizes happen at random

W4-M19 [why] Growing by a factor of **1.5** instead of 2:

- **a)** makes append amortised $O(n)$
- **b)** makes append amortised $O(\log n)$
- **c)** keeps append amortised $O(1)$, with more copies but less wasted space
- **d)** removes the need for resizing

W4-M20 [how] With doubling and no shrinking, just after a resize, about how much of the capacity is unused?

- **a)** none
- **b)** a quarter
- **c)** three quarters
- **d)** about half

W4-M21 [what] Which structure in `dsa/` stores its items in the student's own `DynamicArray`?

- **a)** `Stack`
- **b)** `CircularQueue`
- **c)** `OpenAddressingHashMap`
- **d)** `BinarySearchTree`

W4-M22 [how] In the accounting ("bank") proof for doubling, how many coins must each append be charged?

- **a)** 1
- **b)** 3
- **c)** $\log_2 n$
- **d)** n

Part B — Short answer and essay

W4-E1 [why] (5 marks) Define amortised cost. Prove that n appends to a dynamic array that doubles cost $O(n)$ in total, and conclude the amortised cost of one append.

W4-E2 [why] (4 marks) Explain why growing the capacity by a constant number of slots gives $\Theta(n^2)$ total cost for n appends, however large the constant, while growing by a constant factor gives $\Theta(n)$.

W4-E3 [why] (3 marks) append is "worst-case $O(n)$ and amortised $O(1)$ ". Explain both statements, why both are true, and when the worst case still matters.

W4-E4 [why] (4 marks) Explain thrashing with a concrete sequence of operations, and the shrinking rule that prevents it.

W4-E5 [why] (3 marks) Growth factor is a time–space trade-off. Explain the trade-off, and why CPython’s `list` uses a small factor (about 1.125).

Part C — Trace the growth

Doubling, starting empty with capacity 1, unless stated otherwise. Count one unit of work per element **written** by `append` and one per element **copied** by a `resize`.

W4-T1 [how] After 10 appends: what is the capacity, how many resizes have happened, and how many elements have been copied in total?

W4-T2 [how] For 20 appends, list which appends trigger a `resize`, and compute the total work (writes + copies) and the average work per `append`.

W4-T3 [how] Now the capacity grows by **+3** at each `resize` (1, 4, 7, 10, ...). For 12 appends: which appends trigger a `resize`, what is the final capacity, and how many elements are copied?

W4-T4 [how] A shrinking dynamic array doubles when full and, **after** a `pop`, halves when `size` $\leq$ `capacity` // 4 (and `capacity` > 1). Starting empty, perform `append(1) ... append(5)`, then `pop()` four times. Give the `size` and `capacity` after every operation.

W4-T5 [how] Among the first 40 appends with doubling, which ones are “expensive” (trigger a `resize`)? What pattern do their numbers follow?

Part D — Proofs

W4-P1 [why] (4 marks) A dynamic array grows by a factor of **4** (capacity 1, 4, 16, 64, ...). Prove that n appends cost $O(n)$ in total.

W4-P2 [why] (4 marks) A dynamic array grows by a constant c slots. Prove that n appends copy at least $\frac{n^2}{2c} - n$ elements, and hence that the total cost is $\Omega(n^2)$ for any fixed c .

Part E — Find and fix the bug

Each method belongs to a `DynamicArray` with `_block` (an `Array`), `_size` and `_capacity`.

W4-B1 [how]

```
def append(self, value):
    if self._size > self._capacity:
        self._resize(self._capacity * 2)
    self._block[self._size] = value
    self._size += 1
```

W4-B2 [how]

```
def __getitem__(self, index):
    if index < 0:
        index += self._size
    if not 0 <= index < self._capacity:
        raise IndexError(index)
    return self._block[index]
```

W4-B3 [why] Passes every test except `test_doubling_keeps_resizes_logarithmic`.

```
def append(self, value):
    if self._size == self._capacity:
        self._resize(self._capacity + self.growth)
    self._block[self._size] = value
    self._size += 1
```

W4-B4 [why] Passes every test, yet holds on to memory it should not.

```
def pop(self, index=-1):
    if self._size == 0:
        raise IndexError("pop from empty")
    if index < 0:
        index += self._size
    value = self._block[index]
    for i in range(index, self._size - 1):
        self._block[i] = self._block[i + 1]
    self._size -= 1
    return value
```

Part F — Write the code

In `practice/week04.py`; check with `pytest tests/test_practice_week04.py -v`.

W4-C1 [how] `capacity_after(n, factor=2)` — the capacity after `n` appends.

W4-C2 [how] `copies_for(n, grow)` — how many elements `n` appends copy, when `grow(capacity)` gives the new capacity. Use it to check the numbers in Lecture 04.

W4-C3 [why] `ShrinkingArray` — a dynamic array on an `Array` that doubles when full and halves when a `pop` leaves it one quarter full. Why does `test_shrinking_array_does_not_thrash` pass with the quarter rule and fail with a half rule?